

Framework para Pagamentos na Web

Ana Carolina. O. Souza¹, Bruno F.C. Nascimento¹, Livia M. O. Silva¹, Maurício V.D. Junior¹.

¹ Mestrando em Modelagem Computacional – Universidade Federal de Alagoas (UFAL) – Maceió – AL – Brazil

{bruno.f.c.nascimento, livinhaomena, mauriciodias.junior}@gmail.com
acarolina_os@hotmail.com

Resumo: O presente trabalho descreve o desenvolvimento de um framework para formas de pagamentos, que pode ser implementado em diversos tipos de sistemas web, como: farmácias virtuais, lojas de livros virtuais e etc. Este framework foi desenvolvido a partir do padrão de projeto template method. Foi utilizada a linguagem de programação orientada a objetos Java (versão 6.0) da Sun Microsystem(<http://java.sun.com>), a qual foi escolhida por se tratar de uma linguagem que atua em qualquer plataforma, assim como, aborda de maneira clara os conceitos de Interface, Componente e Framework. Foi utilizada também a IDE Eclipse(versão 3.4.2), produzida pela Apache Software Foundation (<http://www.apache.org/>).

Abstract: The current work shows a framework implementation for Web Payments forms, which could be deployed in many Web software systems, as: Virtual Pharmacies, Virtual Bookstores, Virtual shops, etc. This Framework was developed based on Template Method design pattern. The oriented object java (version 6.0) of Sun Microsystem (<http://java.sun.com>) was used, It was chosen because it is the language that works in many platforms, and It deals the concepts of Interface, Component and Framework clearly. The IDE Eclipse (version 3.4.2), produced by Apache Software Foundation (<http://www.apache.org/>) was used as development platform.

1. Introdução:

O desenvolvimento de software é um processo complexo e consumidor de recursos financeiros e humanos [PFAFFENSELLER, 2001]. Como o escopo dos sistemas tem se tornado cada vez maior, há a necessidade de buscar mecanismos que reduzam sua complexidade. No contexto da engenharia de software, diferentes abordagens buscam melhorar a qualidade dos artefatos de software, bem como diminuir o tempo e o esforço necessário para produzi-los.

A reutilização de artefatos de software é um dos principais argumentos em favor da abordagem de orientação a objetos. Em muitos casos, porém, se constitui uma perspectiva frustrada, pois a reutilização não é característica inerente da orientação a objetos, mas é obtida a partir do uso de técnicas que produzem software reutilizável [PREE, 1994] [RALPH, 1988].

Este artigo tem por objetivo explicar e demonstrar o desenvolvimento de um framework para pagamentos web que possibilite o reuso, do mesmo, em diversas aplicações, caracterizando o reuso de projeto e código. A adoção destas práticas no desenvolvimento de software (em conjunto ou isoladamente) pode diminuir

significativamente o tempo para o desenvolvimento de novas aplicações, bem como a quantidade de código a ser implementado, na medida que parte das responsabilidades das novas aplicações são atribuídas a artefatos de software reutilizados [SILVA, 2000].

O contexto deste artigo será apresentado de acordo com os tópicos: 2 Framework, 2.1 – Framework para pagamento web, 2.2 Modelagem, 2.3 Padrões de Projeto, 2.4 Template Method, 3 Considerações Finais e 4 Referências Bibliográficas.

2. Framework

O Framework é uma camada de trabalho que esconde detalhes específicos de implementação acerca de um limitado universo de atuação, reduzindo a complexidade de interação do utilizador sobre este universo.

Algumas das facilidades marcantes trazidas pelo paradigma de programação orientação a objeto como: ligação dinâmica, polimorfismo e herança, tornam possível a criação deste framework. Estas técnicas foram utilizadas muitas vezes como instrumentos de reutilização, abstração e especialização dos objetos.

- **Orientação a Objeto**

O termo orientação a objetos sugere uma associação entre (abstrações de) “coisas” do mundo real e trechos de programas de computador ou objetos [YOURDON, 1999]. Na programação orientada a objetos (POO), dados e procedimentos passam a fazer parte de um elemento básico, o objeto. A POO introduz uma abordagem na qual o desenvolvedor visualiza seu programa em execução como uma coleção de objetos cooperantes que se comunicam por meio de troca de mensagens [VINCENZI, 2004].

A grande dificuldade na criação de um framework é à disposição das classes de maneira que elas comuniquem dinamicamente. Caso se decida migrar para este paradigma, partindo de trabalhos onde não foi gasto o tempo devido na análise de requisitos e projeto, torna-se necessário redesenhar totalmente a aplicação para que ela suporte esta intensa integração. Reforçando a prática que padrões de projeto fazem parte da documentação indispensável de um framework, e através deles é que se remodela um sistema.

2.1 Framework para pagamentos na web

O framework para pagamento na WEB foi desenvolvido para atender as necessidades de clientes que desejam expandir e melhorar os serviços oferecidos, como a possibilidade de pagamento via WEB.

O Framework do tipo híbrido, que utiliza as características dos frameworks de caixa preta e caixa branca, foi utilizado por possuir características de composição (funcionalidades prontas) e herança (modificar ou estender outras funcionalidades). Uma funcionalidade existente é reutilizada e estendida através de:

- Herança de classes base do framework;
- Sobrecarga de métodos hook predefinidos usando patterns como o Template Method.

Este framework pode ser utilizado por diversos tipos de aplicações web, como: livrarias virtuais, Farmácias virtuais, sites de e-commerce em geral e etc. O administrador que for implementar este Framework poderá escolher entre cinco componentes, que são as formas de pagamentos:

- **Componentes:** Um pacote coerente de artefatos de software que pode ser desenvolvido independentemente e entregue como unidade e que pode ser composto, sem mudança, com outros componentes para construir algo maior.
- o **CBoletoBancario:** Gera um boleto de uma determinada agência bancária. Neste projeto foram definidos dois bancos, o BB (Banco do Brasil) e a CEF (Caixa econômica), porém estes podem ser alterados e até mesmo definir apenas uma única agência para geração de boleto, como mostra o código da figura 1.

```
package framework;

public class CBoletoBancario implements IBoletoBancario{

    public void gerarBoleto(int boleto){
        switch (boleto) {
            case 1:
                //boleto BB
                System.out.println("-----");
                System.out.println("Boleto de Pagamento - BB");
                System.out.println("-----");
                System.out.println("||||||||||||||||");
                break;
            case 2:
                //boleto CEF
                System.out.println("-----");
                System.out.println("Boleto de Pagamento - CEF");
                System.out.println("-----");
                System.out.println("||||||||||||||||");
                break;
            default:
                System.out.println("Não trabalhamos com a forma de pagamento: BOLETO BANCÁRIO");
                break;
        }
    }
}
```

Figura 1 – Código fonte do componente CBoletoBancario.

- o **CCartaoCredito:** Disponibiliza a opção de pagar com cartão de crédito. Neste projeto foram definidos três opções de cartão de crédito: MasterCard, Visa e HiperCard, porém estas opções podem ser alterados e até mesmo definir apenas uma única para a forma de pagamento cartão de crédito, como mostra o código da figura 2.

```
package framework;

public class CCartaoCredito implements ICartaoCredito{

    public void creditarCartao(int cartao){
        switch (cartao) {
            case 1:
                //cartão MasterCard
                System.out.println("-----");
                System.out.println("MasterCard");
                System.out.println("-----");
                System.out.println("$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$");
                break;
            case 2:
                //cartão Visa
                System.out.println("-----");
                System.out.println("Visa");
                System.out.println("-----");
                System.out.println("$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$");
                break;
            case 3:
                //cartão HiperCard
                System.out.println("-----");
                System.out.println("HiperCard");
                System.out.println("-----");
                System.out.println("$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$$");
                break;
            default:
                System.out.println("Não trabalhamos com a forma de pagamento: CARTÃO DE CRÉDITO");
                break;
        }
    }
}
```

Figura 2 – Código fonte do componente CCartaoCredito.

- o **CFinanciamento**: Este componente disponibiliza a opção de financiar compras por empresas de financiamento. Neste projeto foram definidos três opções de Financiamento: iPrev-BB, Fininvest e BPN, porém estas opções podem ser alterados e até mesmo, definir apenas uma única para financiar a compra, como mostra o código da figura 3.

```
package framework;

public class CFinanciamento implements IFinanciamento{

    public void Financiar(int financiamento){
        switch (financiamento) {
            case 1:
                //Financiamento pelo iPrev-BB
                System.out.println("-----");
                System.out.println(" Financiamento pelo iPrev-BB ");
                System.out.println("-----");
                System.out.println("*****");
                break;
            case 2:
                //Financiamento pela Fininvest
                System.out.println("-----");
                System.out.println(" Financiamento pela Fininvest ");
                System.out.println("-----");
                System.out.println("*****");
                break;
            case 3:
                //Financiamento pela BPN
                System.out.println("-----");
                System.out.println(" Financiamento pela BPN ");
                System.out.println("-----");
                System.out.println("*****");
                break;
            default:
                System.out.println("Não trabalhamos com a forma de pagamento: FINANCIAMENTO");
        }
    }
}
```

Figura 3 – Código fonte do componente CFinanciamento.

- o **CTransferenciaEletronica**: Disponibiliza a opção de transferência on-line. Neste projeto foram definidas três opções de transferência: Entre Contas, Poupança para Poupança e Via DOC, porém estas opções podem ser alteradas, e até mesmo definir apenas uma única forma de Transferência on-line, como mostra o código da figura 4.

```
package framework;

public class CTransferenciaEletronica implements ITransferenciaEletronica{

    public void Transferir(int transferencia){
        switch (transferencia) {
            case 1:
                //transferência Entre contas
                System.out.println("-----");
                System.out.println(" Transferência Entre Contas ");
                System.out.println("-----");
                System.out.println("=====");
                break;
            case 2:
                //transferência Poupança p/ Conta
                System.out.println("-----");
                System.out.println(" Transferência Poupança p/ Conta");
                System.out.println("-----");
                System.out.println("=====");
                break;
            case 3:
                //transferência DOC
                System.out.println("-----");
                System.out.println(" Transferência via DOC ");
                System.out.println("-----");
                System.out.println("=====");
                break;
            default:
                System.out.println("Não trabalhamos com a forma de pagamento: TRANSFERÊNCIA ELETRÔNICA");
        }
    }
}
```

Figura 4 – Código fonte do componente CTransferenciaEletronica.

- o **CBilhetePremiado**: Concede desconto nas compras. Disponibiliza a forma de ter desconto, ou não, em compra realizada, como mostra o código da figura 5.

```
package framework;

public class CBilhetePremiado implements IBilhetePremiado{

    public void TrocarBilhete(int bilhete){
        switch (bilhete) {
            case 1:
                //Desconto no produto
                System.out.println("-----");
                System.out.println(" Desconto no Produto ");
                System.out.println("-----");
                System.out.println("-----");
                break;

            case 2:
                //Pagamento Total
                System.out.println("-----");
                System.out.println(" Pagamento Total");
                System.out.println("-----");
                System.out.println("-----");
                break;

            default:
                System.out.println("Não trabalhamos com a forma de pagamento: BILHETE PREMIADO");
                break;
        }
    }
}
```

Figura 5 – Código fonte do componente CBilhetePremiado.

O framework possui dois pontos de adaptação (hotspots), que expressam aspectos do domínio do framework e não podem ser antecipados, ou seja, são dependentes da aplicação, que são os métodos cabeçalho () e rodapé () figura 6.

```
package framework;

public abstract class FrameworkPagamentoWeb {

    //declaração das interfaces dos componentes
    public IBoletoBancario IBoleto;
    public ICartaoCredito ICredito;
    public ITransferenciaEletronica ITrEletronica;
    public IFinanciamento IFinanciamento;
    public IBilhetePremiado IBilhetePremiado;

    public FrameworkPagamentoWeb (int boleto,int cartao,int transferencia,int financiamento,int bilhete) {
        //chamada das interfaces dos componentes
        IBoleto = new CBoletoBancario();
        ICredito = new CCartaoCredito();
        ITrEletronica = new CTransferenciaEletronica();
        IFinanciamento = new CFinanciamento();
        IBilhetePremiado = new CBilhetePremiado();

        controla_forma_pagamento (boleto,cartao,transferencia,financiamento,bilhete);
        //aqui é declarado o método TEMPLATE
    }

    //pontos de adaptação (hotspots)
    abstract void cabeçalho();
    abstract void rodapé();

    private void controla_forma_pagamento(int boleto,int cartao,int transferencia,int financiamento, int bilhete) {

        //aqui é definido o controle (Método Template)
        cabeçalho();
        IBoleto.gerarBoleto(boleto);
        ICredito.CreditarCartao(cartao);
        ITrEletronica.Transferir(transferencia);
        IFinanciamento.Financiar(financiamento);
        IBilhetePremiado.TrocarBilhete(bilhete);
        rodapé();
        System.out.println("");
        System.out.println("");
    }
}
```

Figura 6 – Código Fonte do Framework com os hotspots cabeçalho e rodapé

- **Interface:** Uma declaração de interface introduz um novo tipo de referência, cujos membros são classes, interfaces, constantes e métodos abstratos. Esses tipos não possuem implementação, por outro lado, as suas classes relacionadas (ou subclasses) fornecem a implementação dos métodos abstratos [JAVA, 05].

- o **IBilhetePremiado:** Nesta interface o método abstrato TrocarBilhete (int boleto) é definido e apenas será implementado no componente CbilhetePremiado. Se houver a necessidade de mudar o comportamento deste método, o framework e a interface não serão modificados, pois apenas o componente que implanta o método sofrerá alteração. A figura 7 mostra o código fonte do framework em questão:

```
package framework;  
  
public interface IBilhetePremiado {  
    public void TrocarBilhete(int bilhete);  
}
```

Figura 7 – Código fonte da interface IBilhetePremiado.

- o **IBoletoBancario:** nesta interface é definido o método abstrato GerarBoleto (int boleto) do componente CboletoBancario. A figura 8 mostra o código fonte da interface IBoletoBancario:

```
package framework;  
  
public interface IBoletoBancario {  
    public void gerarBoleto(int boleto);  
}
```

Figura 8 – Código fonte da interface IBoletoBancario.

- o **ICartaoCredito:** O método abstrato CreditarCartao(int cartao) é definido nesta interface e implementado no componente CCartaoCredito, se o usuário desejar ele poderá acrescentar os cartões que serão permitidos fazer pagamentos na WEB, expandindo as possibilidades de pagamento com cartão de crédito. A figura 9 mostra o código fonte da interface ICartaoCredito:

```
package framework;  
  
public interface ICartaoCredito {  
    public void CreditarCartao(int cartao);  
}
```

Figura 9 – Código fonte da interface ICartaoCredito.

- o **IFinanciamento:** Nesta interface o método abstrato Financiar (int financiancia) é definido e implementado no componente CFinanciamento, neste componente também é possível expandir as formas de financiamento sem que o framework e a interface sejam

alteradas. A figura 10 mostra o código fonte da interface IFinanciamento:

```
package framework;

public interface IFinanciamento {

    public void Financiar(int financiam);

}
```

Figura 10 – Código fonte da interface IFinanciamento.

- o **ITranferenciaEletronica:** O método abstrato Transferir (int transfere) é definido e implementado no componente CTransferenciaEletronica. A figura 11 mostra o código fonte da interface ITransferenciaEletronica:

```
package framework;

public interface ITransferenciaEletronica {

    public void Transferir(int transfere);

}
```

Figura 11 – código fonte da interface ITransferenciaEletronica.

2.2 Modelagem

A modelagem do sistema foi desenvolvida no Enterprise Architect 7.5 e consiste dos diagramas de Classes, Casos de uso, Componentes e Atividade a seguir:

- A figura 12 mostra o diagrama de classes do framework Forma de Pagamento WEB,

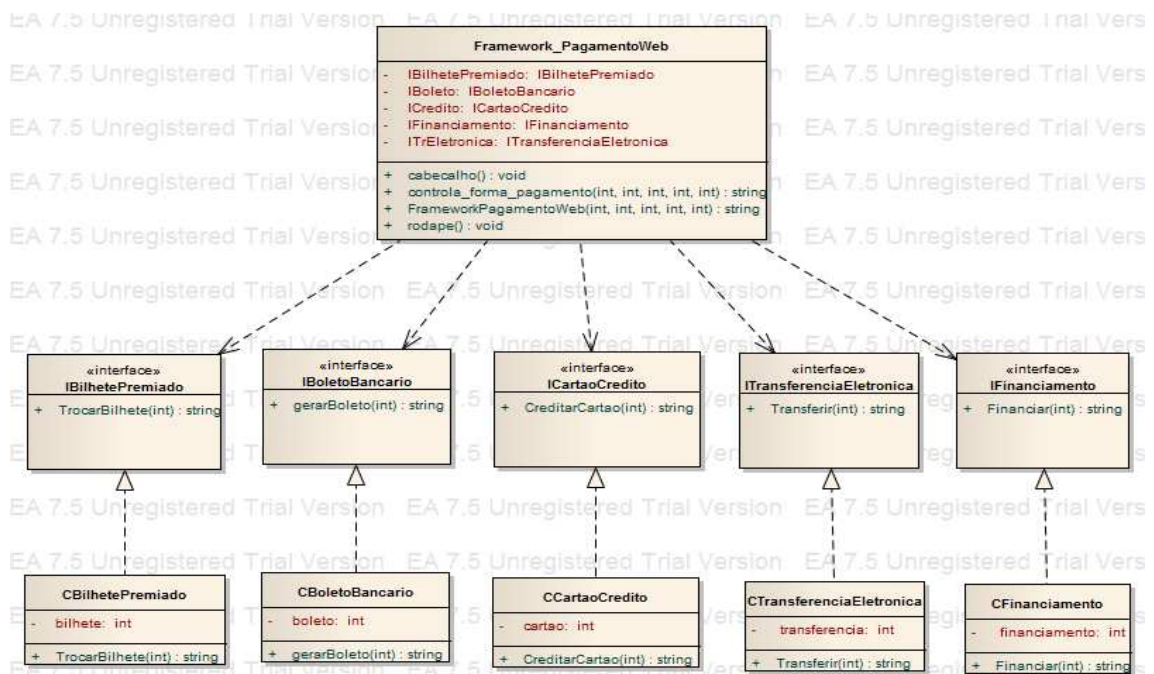


Figura 12 – Framework forma de Pagamento WEB.

- A figura 13 mostra o diagrama de caso de uso:

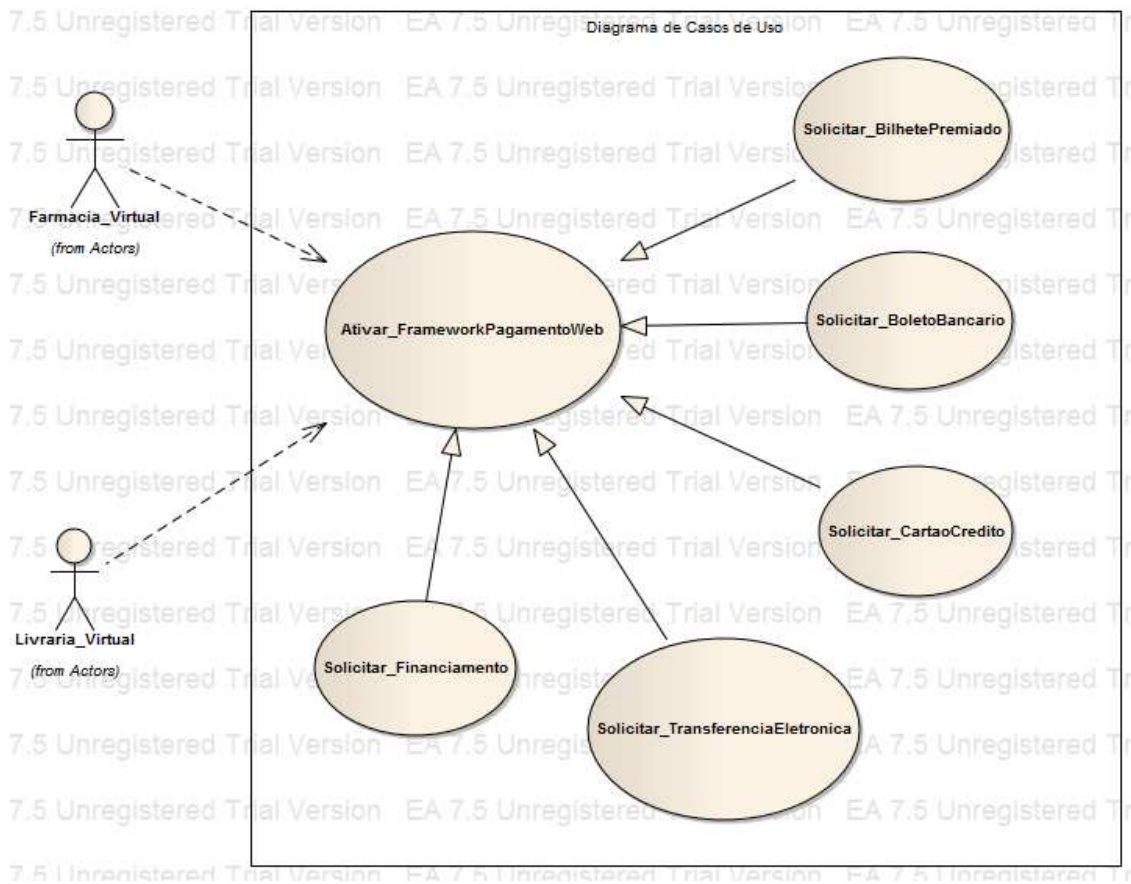


Figura 13 – Diagrama de casos de uso.

- A figura 14 mostra o diagrama de componentes:

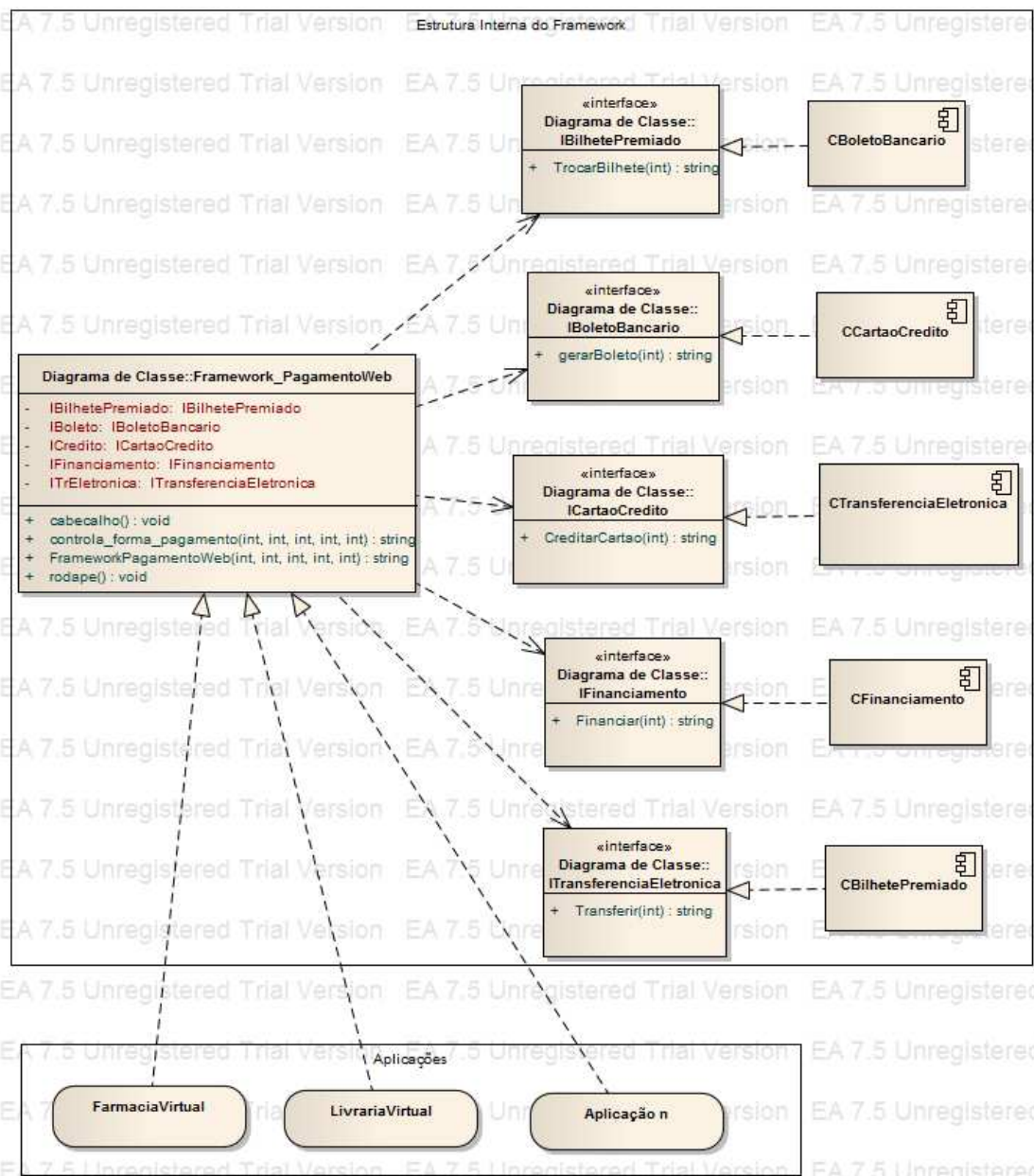


Figura 14 – Diagrama de componentes.

A figura 15 e 16 mostram, o diagrama de atividade:

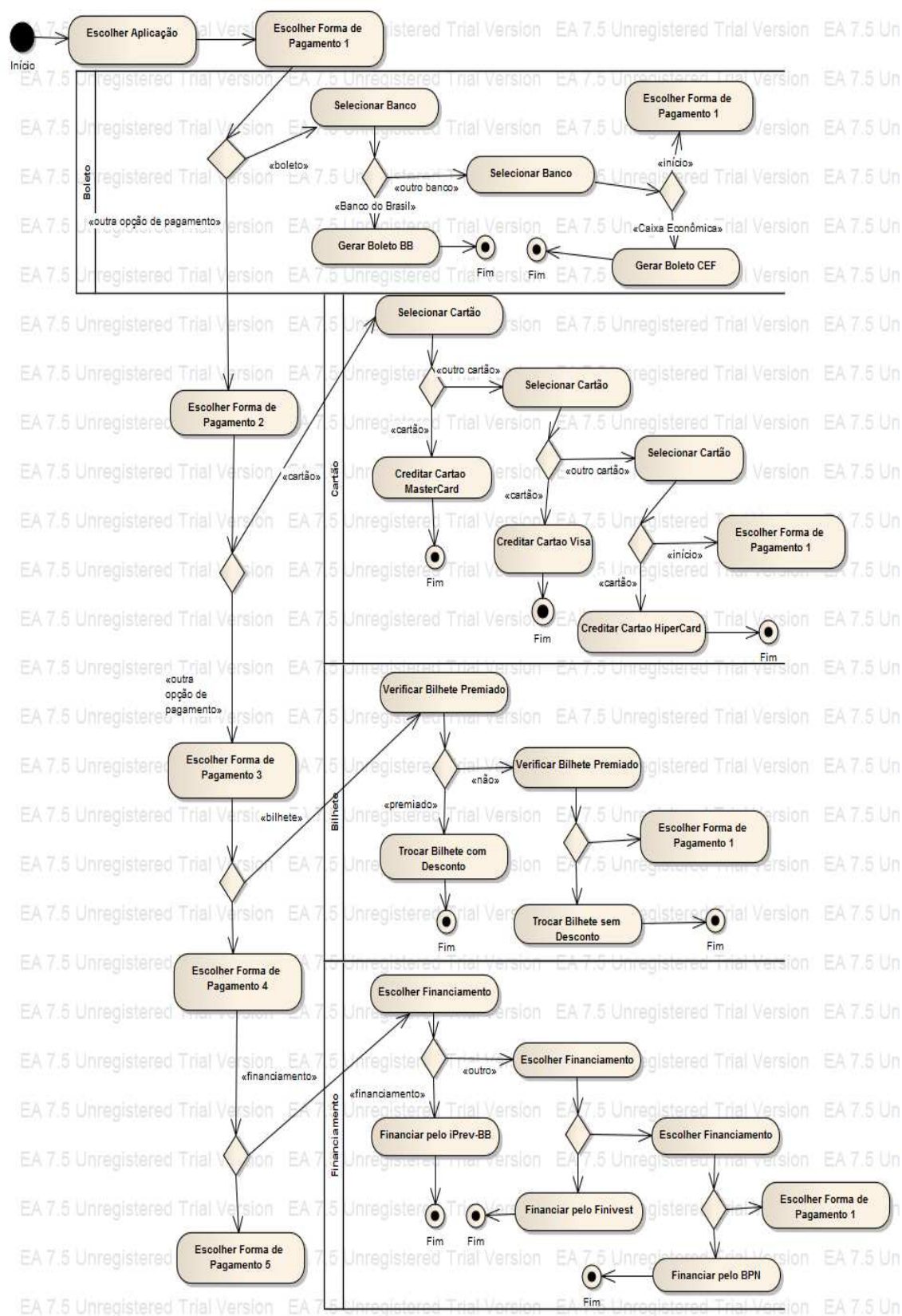


Figura 15 – Diagrama de atividades.

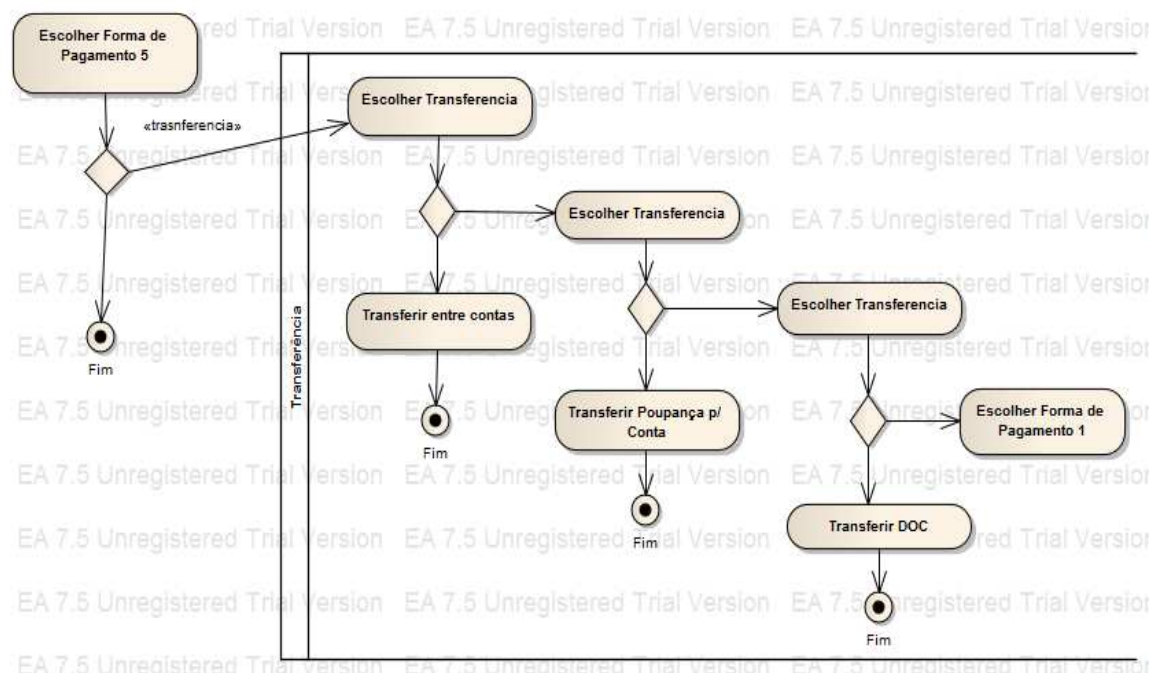


Figura 16 – Diagrama de atividades (continuação).

2.3 Padrões de Projeto

Segundo Alexander et. all (1977) cada padrão descreve um problema no nosso ambiente e o núcleo da sua solução, de tal forma que você possa usá-lo milhões de vezes, sem nunca utilizá-lo do mesmo modo.

Um padrão é uma entidade que documenta um problema recorrente, uma solução e a situação em que deve ser aplicado. Os padrões podem ser utilizados em diversas etapas do desenvolvimento de software. Um dos benefícios apresentados por [VLISSIDES 1996] é a possibilidade de armazenar experiências, tornando-as acessíveis aos não experientes.

No Projeto de um Framework para pagamentos web, foi utilizado o padrão de projeto Template Method.

2.4 Template Method

É um padrão que define o esqueleto de um algoritmo para ser utilizado posteriormente pelas classes que herdarão aquele modelo abstrato. Permite que as subclasses redefinam comportamentos de sua classe pai, de acordo com a sua realidade. Sua tradução para o português pode ser entendida como "método do molde".

Motivação: Ela é utilizada para criar blocos comuns de código a diferentes classes uma única vez, fazendo todas estas classes herdarem este comportamento comum da classe template abstrata. As classes filhas vão concretizar o seu comportamento sobrecarregando os métodos da classe pai através de polimorfismo. As consequências da utilização desta técnica são o aumento da reutilização de código, mas em contrapartida aumenta o acoplamento gerado pela herança, implementada entre a classe abstrata e as classes filhas. Esta prática é a base para a construção de frameworks, ver Figura 17.

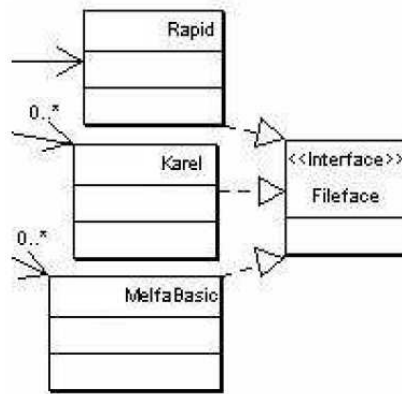


Figura 17 – Exemplo de utilização do método template.

3 Considerações Finais

- **Experiência da equipe**

A abordagem de framework para o presente trabalho trouxe maior flexibilidade para o desenvolvimento do mesmo, onde os detalhes de implementações ficaram escondidos nos componentes: CBilhetePremiado, CBoletoBancario, CCartaoCredito, CTransferenciaEletronica e CFinaciamento.

A utilização de Framework permite que mais componentes possam ser desenvolvidos para outras aplicações sem que grandes alterações sejam feitas na camada, assim como, componentes podem deixar de ser usados caso uma aplicação não requeira todas presentes no projeto.

Como exemplos foram desenvolvidas duas aplicações Farmácia Virtual e Livraria Virtual, cada aplicação pode conter todas ou apenas algumas formas de pagamentos, ou seja, formas de pagamento específicas requeridas pelo cliente.

A figura 18 mostra a tela da aplicação Livraria Virtual do sistema, onde o usuário poderá especificar quais formas de pagamentos deseja na sua aplicação e as características da forma de pagamento, por exemplo, se o cliente desejar a forma de pagamento Cartão de Crédito, ele poderá optar por Visa, Mater ou Hipercard.

Escolha a Aplicação :		LivrariaVirtual
☒ Boleto Bancário - [1] BB [2] CEF		1
☒ Cartão Crédito - [1] Master Card [2] Visa [3] HiperCard		2
☒ Transferência - [1] Entre Contas [2] Poupança p/ Conta [3] via DOC		3
☒ Financiamento - [1] IPREV-BB [2] FINIVEST [3] BPN		1
☒ Bilhete Premiado - [1] Desconto Produto [2] Pagamento Total		2

-- <Inicio> Livraria Virtual [Livros 24horas] </Inicio> --

Boleto de Pagamento - BB

CartaoCredito - Visa

Transferência via DOC

OK

A figura 18 – Mostra que o cliente da Livraria Virtual optou por todas as formas de pagamento disponíveis.

A segunda aplicação desenvolvida foi a Farmácia virtual, que segue o mesmo padrão da Livraria Virtual, onde o cliente poderá optar por quais formas de pagamento WEB deseja.

Baseando-se nos dois exemplos acima citados, podemos perceber que o presente projeto pode ser expandido para várias aplicações, que utiliza formas de pagamento na WEB. Esta flexibilidade traz maior agilidade na construção de novas aplicações e conseqüentemente maior produtividade, pois com menos tempo e esforço poderemos construir aplicações específicas

4 Referências Bibliográfica

- [ALEXANDER, 77] Alexander, C., Ishikawa, S., and Silverstein, M. (1977). A Pattern Language. Oxford University Press, 1 edition.
- [JAVA, 05] The java Language Specification, Third Edition 2005 – Chapter 9. (http://java.sun.com/docs/books/jls/third_edition/html/interfaces.html)
- [PFAFFENSELLER, 01] Pfaffenseller, M., Pfaffenseller, M., and Kroth, E. (2001). Ferramenta de apoio ao desenvolvimento de software baseado em componentes. I Congresso Brasileiro de Computação.
- [PREE, 94] Pree, W. (1994). Design patterns for object oriented software development. Addison -Wesley.
- [RALPH, 88] Ralph E. Johnson, B. F. (1988). Designing reusable classes. Journal of Object-Oriented Programming. SIGS (June/July. 1988), pages 22–35.
- [SILVA, 00] Silva, R. P. (2000). Suporte ao desenvolvimento e uso de frameworks e componentes. Universidade Federal do Rio Grande do Sul - Programa de Pós-Graduação em Computação.
- [VINCENZI, 04] Vincenzi, A. M. R. (2004). Orientação a objeto: definição, implementação e análise de recursos de teste e validação. Instituto de Ciências Matemáticas e de Computação - ICMC - USP, pages 28–29.
- [VLISSIDES, 0] Vlissides, erns: The top ten misconceptions.
- [YOURDON, 99] Yourdon, E. (1999). Análise e projeto orientados a objetos. Makron Books, São Paulo.